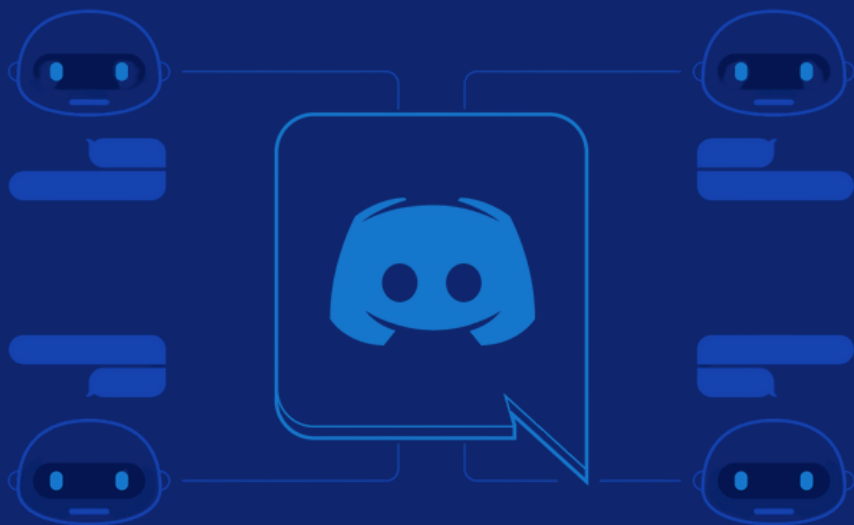


# CRIANDO SEU BOT DO DISCORD

## GUIA DE DESENVOLVIMENTO



<Designed by Danilo Marley>



# SUMÁRIO

## 1. INTRODUÇÃO À TECNOLOGIA

O QUE É DISCORD, BOTS E POR QUE USAR NODE.JS.

## 2. FERRAMENTAS PARA DESENVOLVIMENTO

INSTALAÇÃO E CONFIGURAÇÃO DO AMBIENTE

## 3. PASSO A PASSO DE DESENVOLVIMENTO

CRIANDO O BOT E COMANDOS BÁSICOS

## 4. CONCLUSÃO E REFLEXÃO

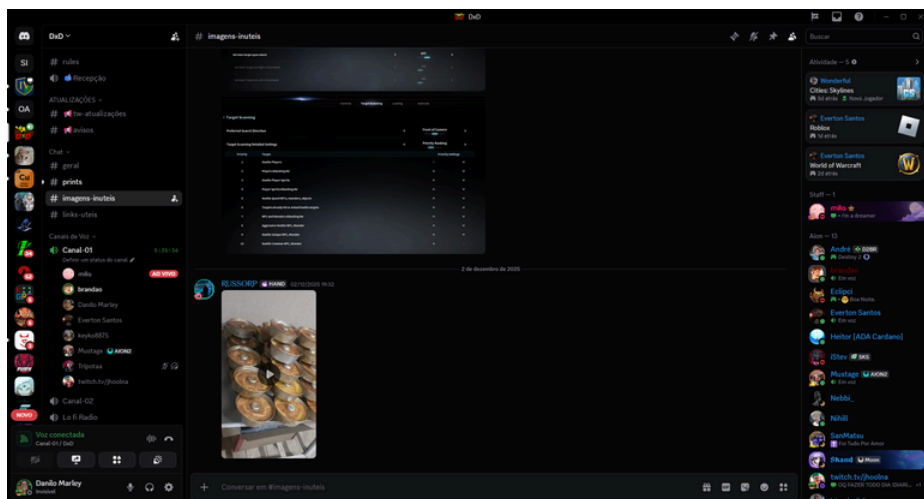
APRENDIZADOS E PRÓXIMOS PASSOS



# INTRODUÇÃO À TECNOLOGIA

## O QUE É DISCORD?

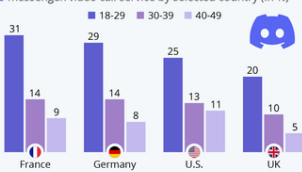
Discord é uma plataforma de comunicação que permite comunicação por texto, voz e vídeo através de servidores organizados em canais. Lançado em 2015, é amplamente usado por comunidades de gamers, desenvolvedores e diversos grupos.



\*Descrição da imagem: Tela da Discord mostrando a barra lateral com servidores, lista de canais à esquerda e área de chat principal com mensagens.\*

### Discord Is a Firm Favorite Among Younger Audiences

Share of adults that use Discord regularly as an instant messenger/video call service by selected country (in %)



Basic respondents who use instant messengers regularly  
At least 980 adults per country surveyed online between April - March 2022  
Source: Statista Global Consumer Survey



statista



### STATS OVERVIEW



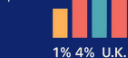
INTERNATIONAL  
70%+ of users  
outside the U.S.

BRAZIL 4-5%  
INDIA 4-5%  
U.K. 4%  
CANADA 3-4%



150 COUNTRIES

WEBSTAFF TRAFFIC  
18+  
visits  
per month

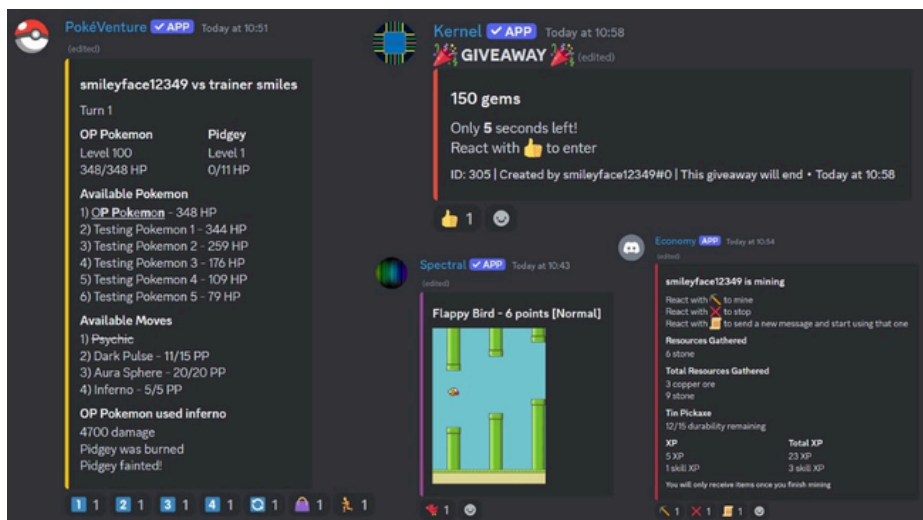


# INTRODUÇÃO À TECNOLOGIA

## O QUE SÃO BOTS DO DISCORD?

**Bots são aplicações automatizadas que executam tarefas no Discord sem intervenção humana constante. Eles podem:**

- Responder a comandos
- Moderar servidores
- Fornecer informações
- Criar interações (enquetes, jogos)



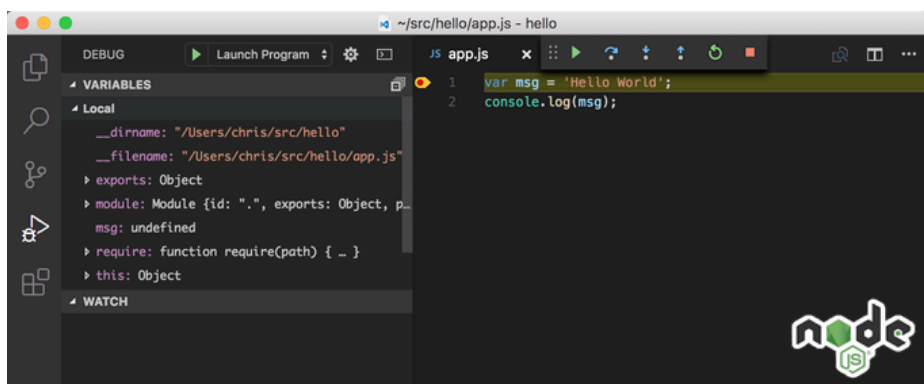
*\* Descrição da imagem: Exemplos de interações de bots, o primeiro mostra um bot que simula a luta entre dois treinadores pokemons, na segunda mostra um sorteio feito em um servidor do discord, na terceira mostra a pontuação de um usuário no discord, na quarta mostra um minigame de mineração.\**



# INTRODUÇÃO À TECNOLOGIA

## POR QUE NODE.JS E DISCORD.JS?

**Node.js** é um ambiente de execução JavaScript que permite executar JavaScript no servidor. É amplamente usado e tem grande ecossistema de pacotes.



\* Descrição da imagem: Exemplos projeto inicial para aprensetar a mensagem "Hello World" ao usuário \*

**Discord.js** é a biblioteca JavaScript mais popular para criar bots do Discord. Oferece:

- Documentação excelente
- Suporte completo às funcionalidades do Discord
- Sistema de eventos intuitivo
- Comunidade muito ativa

```
1  const Discord = require('discord.js');
2  const client = new Discord.Client();
3
4  client.once('ready', () => {
5    console.log('Ready!');
6  });
7
8  client.login('---');
9
10 client.on('message', message => {
11   console.log(message.content);
12 });
13
14 if (message.content === '!Spaghetti') {
15   message.channel.send('Bolognese !');
16 }
```

\* Descrição da imagem: Projeto exemplo de bot para discord inicializando e ao receber a mensagem !Spaghetti retorna ao usuário Bolognese\*



# INTRODUÇÃO À TECNOLOGIA

## O QUE VAMOS CONSTRUIR?

Neste ebook, criaremos um  
**Bot de Utilidades do Servidor**

**Comandos de Informação:** */ping, /help*

**Comandos de Interação:** */joke*

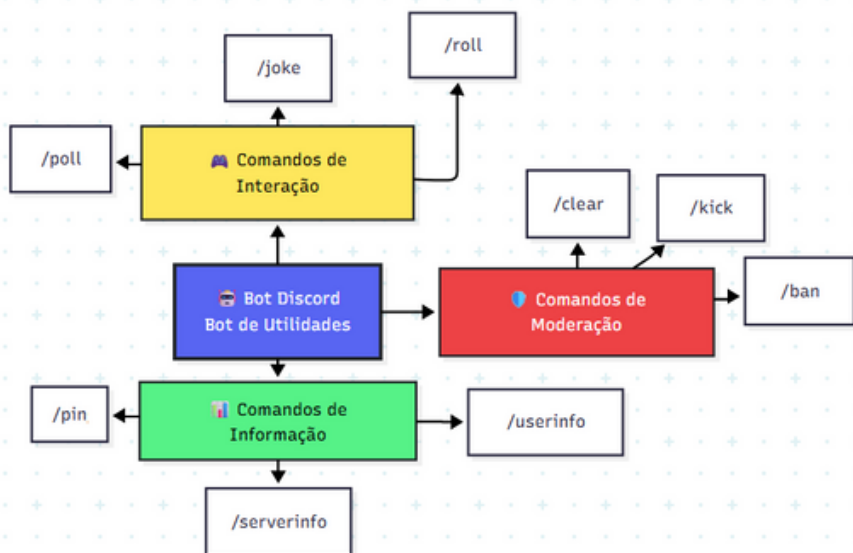


Diagrama em blocos mostrando o bot no centro conectado a três categorias de comandos (Informação, Moderação, Interação) com exemplos de cada tipo.

# FERRAMENTAS PARA O DESENVOLVIMENTO

## INSTALAÇÃO DAS FERRAMENTAS

### Node.js e npm

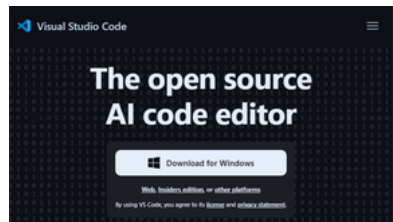
1. Acesse <https://nodejs.org/>
2. Baixe a versão **\*\*LTS\*\*** (recomendada)
3. Execute o instalador
4. Verifique a instalação no terminal:



*\*Descrição da imagem: Pagina inicial para download do Node.JS*

### Visual Studio Code

1. Acesse <https://code.visualstudio.com/>
2. Baixe e instale
3. Extensões úteis: Prettier, ESLint, GitLens

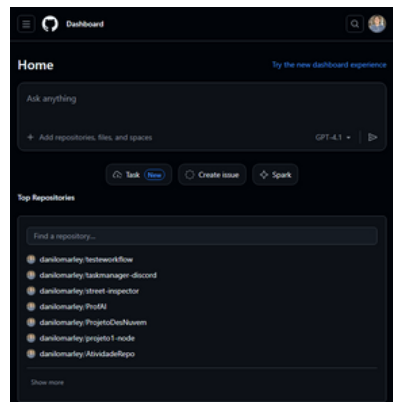


*\*Descrição da imagem: Pagina inicial para download do visual studio code\**

### Git e GitHub

1. Git: Acesse <https://git-scm.com/downloads> e instale
2. GitHub: Crie conta em <https://github.com>
3. Configure Git  
No bash digite os comandos abaixo:

```
bash
git config --global user.name "Seu Nome"
git config --global user.email "seu.email@exemplo.com"
```



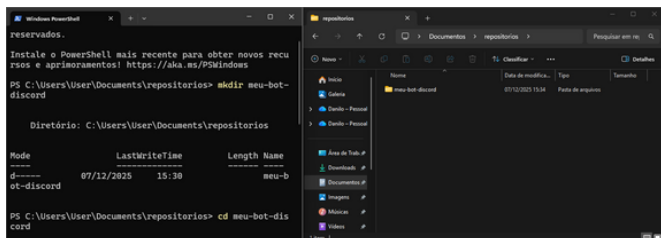
*\*Descrição da imagem: Pagina inicial do GitHub*

# FERRAMENTAS PARA O DESENVOLVIMENTO

## CONFIGURAÇÃO DO PROJETO

### 1. Criar pasta do projeto

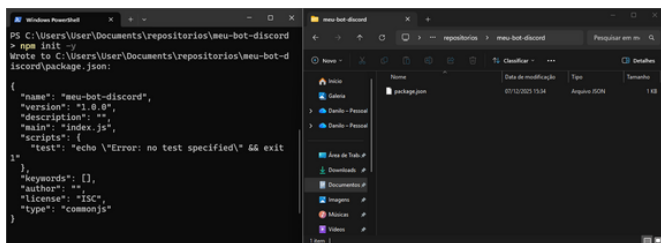
```
mkdir meu-bot-discord  
cd meu-bot-discord
```



\*Descrição da imagem: Bash mostrando execução do comando e resultado no explorer do windows

### 2. Inicializar projeto Node.js

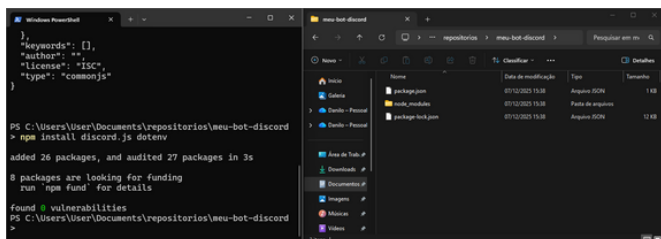
```
npm init -y
```



\*Descrição da imagem: Bash mostrando execução do comando e resultado no explorer do windows

### 3. Instalar dependências

```
npm install discord.js dotenv
```



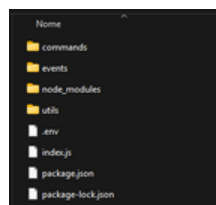
\*Descrição da imagem: Bash mostrando execução do comando e resultado no explorer do windows

### 4. Estrutura de pastas

Crie a seguinte estrutura:

\*Descrição da imagem: Visualização hierárquica da estrutura de pastas do projeto com ícones de pastas e arquivos, mostrando commands, events, utils e arquivos principais.\*

```
meu-bot-discord/  
├── commands/  
├── events/  
├── utils/  
├── index.js  
├── package.json  
└── .env
```

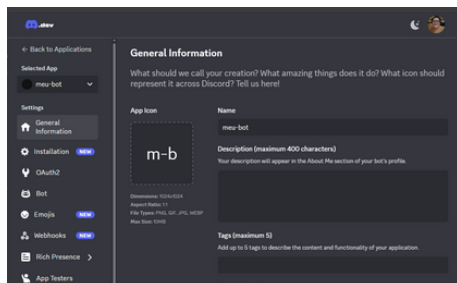


# FERRAMENTAS PARA O DESENVOLVIMENTO

## DISCORD DEVELOPER PORTAL

### Criar Aplicação e Bot

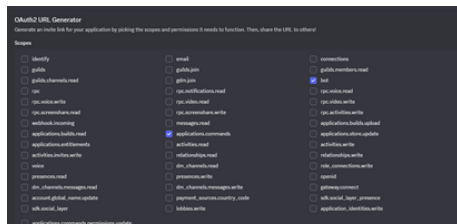
1. Acesse <https://discord.com/developers/applications>
2. Clique em "New Application"
3. Dê um nome e clique em "Create"
4. Vá em "Bot" e clique em "Add Bot"
5. Em "Bot" > Token, clique em "Reset Token" e copie o token
6. **⚠️ IMPORTANTE:** Guarde este token em local seguro!



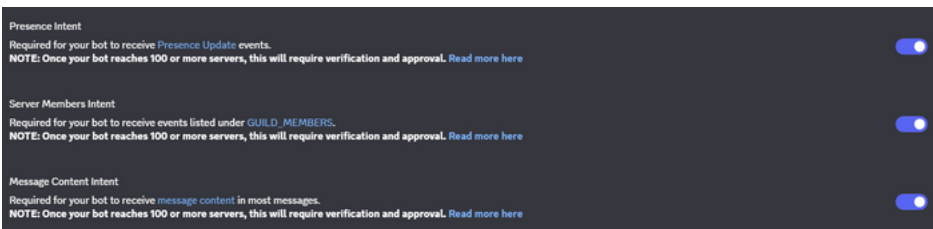
\*Descrição da imagem: Interface do Discord Developer Portal com formulário de criação de aplicação, campos para nome, menu lateral mostrando opções disponíveis.\*

## Configurar Permissões

1. Vá em "OAuth2" > "URL Generator"
2. Em "Scopes", selecione: **bot** e **applications.commands**
3. Em "Bot Permissions", selecione as permissões necessárias
4. Copie a URL gerada e use para adicionar o bot ao servidor
5. Na aba "Bot" ative as tres opções **Presence Intent**, **Server Members Intent** e **Message Content Intent**.



\*Descrição da imagem: Interface mostrando seção de Scopes com bot e applications.commands marcados, e seção de Bot Permissions com várias permissões selecionadas (Send Messages, Manage Messages, etc.).\*



\*Descrição da imagem: Configuração da aba bot para liberar o acesso ao bot a receber eventos, mensagens e conteúdos.

# FERRAMENTAS PARA O DESENVOLVIMENTO

## CONFIGURAÇÕES DO PROJETO

### Configurar arquivo .env

1. Edite o arquivo “.env” criado anteriormente, nele deverá conter as informações:

```
DISCORD_TOKEN=seu_token_aqui  
CLIENT_ID=seu_client_id_aqui
```

O Client ID está em “General Information” > “Application ID”.

O Discord Token foi gerado no começo da configuração (o que foi copiado)



*\*Descrição da imagem: Editor de código mostrando o arquivo .env com duas linhas contendo DISCORD\_TOKEN e CLIENT\_ID com valores de exemplo (token mascarado por segurança).\**

# PASSO A PASSO DE DESENVOLVIMENTO

## BOT BÁSICO – PRIMEIRA CONEXÃO

### 1. Editando arquivo index.js

No seu arquivo index.js adicione o seguinte conteúdo:

```
const { Client, GatewayIntentBits } = require('discord.js');
require('dotenv').config();

const client = new Client({
  intents: [
    GatewayIntentBits.Guilds,
    GatewayIntentBits.GuildMessages,
    GatewayIntentBits.MessageContent
  ]
});

client.once('ready', () => {
  console.log('✅ Bot online! Logado como ${client.user.tag}');
});

client.login(process.env.DISCORD_TOKEN);
```

### 2. Inicie o seu bot

No console do visual studio code execute o comando:

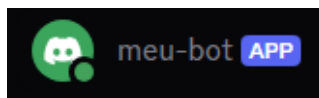
```
node index.js
```

Ao executar o comando o bot será iniciado e você verá a mensagem de sucesso “Bot online!”

```
PS C:\Users\User\Documents\repositorios\meu-bot-discord> node index.js
[dotenv@17.2.3] injecting env (2) from .env -- tip: 🌟 suppress all logs
[dotenv@17.2.3] injecting env (2) from .env -- tip: 🌟 suppress all logs
with { quiet: true }
✅ Bot online! Logado como meu-bot#7462
(node:22092) DeprecationWarning: The ready event has been renamed to clientReady to distinguish it from the gateway READY event and will only emit under that name in v15. Please use clientReady instead.
(Use 'node --trace-deprecation ...' to show where the warning was created)
```

\*Descrição da imagem: Janela do terminal mostrando a execução do comando node index.js e a saída com a mensagem de sucesso indicando que o bot está online com o nome do bot.\*

Você também verá que o seu bot já ficou “online” no seu servidor



# PASSO A PASSO DE DESENVOLVIMENTO

## SISTEMA DE COMANDOS MODULAR

### 1. Atualize `index.js` para carregar comandos automaticamente:

```
const { Client, GatewayIntentBits, Collection } = require('discord.js');
const fs = require('fs');
const path = require('path');
require('dotenv').config();

const client = new Client({
  intents: [GatewayIntentBits.Guilds, GatewayIntentBits.GuildMessages,
    GatewayIntentBits.MessageContent]
});

client.commands = new Collection();

// Carregar comandos
const commandsPath = path.join(__dirname, 'commands');
const commandFiles = fs.readdirSync(commandsPath).filter(file => file.endsWith('.js'));

for (const file of commandFiles) {
  const command = require(path.join(commandsPath, file));
  if ('data' in command && 'execute' in command) {
    client.commands.set(command.data.name, command);
  }
}

// Carregar eventos
const eventsPath = path.join(__dirname, 'events');
const eventFiles = fs.readdirSync(eventsPath).filter(file => file.endsWith('.js'));

for (const file of eventFiles) {
  const event = require(path.join(eventsPath, file));
  if (event.once) {
    client.once(event.name, (...args) => event.execute(...args));
  } else {
    client.on(event.name, (...args) => event.execute(...args));
  }
}

client.login(process.env.DISCORD_TOKEN);
```

\*Descrição da imagem: Editor de código mostrando o código do index.js com destaque nas seções que carregam comandos e eventos automaticamente das pastas.\*

# PASSO A PASSO DE DESENVOLVIMENTO

## CRIANDO COMANDOS

### Comando /ping

#### 1. Crie **commands/ping.js**:

```
const { SlashCommandBuilder, EmbedBuilder } = require('discord.js');

module.exports = {
  data: new SlashCommandBuilder()
    .setName('ping')
    .setDescription('Verifica a latência do bot'),

  async execute(interaction) {
    const sent = await interaction.reply({
      content: '🏓 Calculando ping...',
      fetchReply: true
    });

    const embed = new EmbedBuilder()
      .setTitle('🏓 Pong!')
      .addFields(
        { name: '🕒 Latência da API', value: `${Math.round(interaction.client.ws.ping)}ms` },
        { name: '🕒 Latência da Mensagem', value: `${sent.createdTimestamp - interaction.createdTimestamp}ms` }
      )
      .setColor('#0099ff');

    await interaction.editReply({ content: '', embeds: [embed] });
  }
};
```

### Comando /help

#### 1. Crie **commands/help.js**:

```
const { SlashCommandBuilder, EmbedBuilder } = require('discord.js');

module.exports = {
  data: new SlashCommandBuilder()
    .setName('help')
    .setDescription('Lista todos os comandos disponíveis'),

  async execute(interaction) {
    const embed = new EmbedBuilder()
      .setTitle('📖 Lista de Comandos')
      .addFields(
        { name: '📄 Informação', value: `"/ping" "/serverinfo" "/userinfo" "/avatar"`, },
        { name: '🔊 Moderação', value: `"/clear" "/kick" "/ban"`, },
        { name: '🎭 Interações', value: `"/poll" "/joke" "/8ball" "/roll"`, }
      )
      .setColor('#0099ff');

    await interaction.reply({ embeds: [embed] });
  }
};
```

# PASSO A PASSO DE DESENVOLVIMENTO

## CRIANDO COMANDOS

### Comando /joke (Interação)

#### 1. Crie `commands/joke.js`:

```
const { SlashCommandBuilder, EmbedBuilder } = require('discord.js');

const jokes = [
  'Por que o programador não dorme? Porque ele tem bugs para resolver! 🐛',
  'Qual é o café favorito do programador? Java! ☕',
  'Por que o JavaScript não consegue dormir? Porque ele tem callbacks! 😴',
  'O que um programador faz quando está com fome? Ele come bugs! 🐛',
  'Por que o Python não usa chaves? Porque ele é indentado! 🐍',
  'Qual é a linguagem de programação favorita do pescador? C# (C Sharp)! 🐟',
  'Por que o desenvolvedor não consegue encontrar o erro? Porque ele está no código! 🐞',
  'O que o código disse para o desenvolvedor? "Você me compilou!" 🖥️',
  'Por que o banco de dados quebrou? Porque não tinha backup! 💾',
  'Qual é o animal favorito do programador? O gato (cat)! 🐱'
];

module.exports = {
  data: new SlashCommandBuilder()
    .setName('joke')
    .setDescription('Conta uma piada aleatória'),

  async execute(interaction) {
    const randomJoke = jokes[Math.floor(Math.random() * jokes.length)];

    const embed = new EmbedBuilder()
      .setTitle('😄 Piada do Dia')
      .setDescription(randomJoke)
      .setColor('#0099ff')
      .setTimestamp();

    await interaction.reply({ embeds: [embed] });
  }
};
```

# PASSO A PASSO DE DESENVOLVIMENTO

## CRIANDO COMANDOS

### Comando /joke (Interação)

#### 1. Crie **commands/joke.js**:

```
module.exports = {  
  name: 'ready',  
  once: true,  
  execute(client) {  
    console.log(`✅ Bot online! Logado como ${client.user.tag}`);  
    client.user.setActivity('Desenvolvendo um ebook! 📖', { type: 'Watching' });  
  }  
};
```